MIT ACADEMY OF ENGINEERING

ALANDI, PUNE

WEB TECHNOLOGY

PRACTICAL NO. 3

FRONT-END JAVASCRIPT FRAMEWORKS: REACTJS

Project:

EVENTSPHERE

EVENT & WORKSHOP REGISTRATION PORTAL

Student Name:	Atharv Jadhav
PRN:	202401100102
Branch:	Software Engineering
Division:	B

GitHub Repository : https://github.com/teamstreetup/eventsphere-react-validation

Deployment Link : https://teamstreetup.github.io/eventsphere-react-validation/

1. AIM

To design, develop and deploy a ReactJS single-page web application named EventSphere - Event & Workshop Registration Portal, demonstrating React components, state management using hooks, event handling, client-side form validation, error and success messages, interactive application management, dynamic pricing calculation, and browser-side data persistence.

2. PROBLEM STATEMENT

The practical requires an individual front-end web application developed using ReactJS. The application must demonstrate client-side validation and interactive functionality, use React concepts such as components, state and event handling, display suitable error messages, provide success or confirmation feedback, offer a user-friendly interface, and be tested with valid and invalid inputs. The completed application must also be deployed and submitted through a public link.

EventSphere addresses these requirements through a multi-step conference and workshop registration portal. A user can enter personal details, academic profile, select ticket tiers and workshop tracks, apply promotional coupons with live discount calculation, inspect real-time password security criteria, and validate details before submission. Field-level error messages appear when inputs are invalid, and a verified digital attendee pass with celebratory confetti is generated upon successful submission. Registered attendees are displayed in an interactive directory with search and filter capabilities, and are persisted in browser localStorage so they remain available after a refresh in the same browser.

3. OBJECTIVES

- Develop an individual ReactJS application using a component-based structure.
- Use React state (useState) to manage form values, validation errors, step progression, and attendee records.
- Use React useMemo for real-time subtotal, add-on workshop costs, and promotional discount calculations.
- Implement controlled form inputs and event handlers for user interaction.
- Validate name, email, Indian mobile number, date of birth (enforcing age ≥ 18), institution, portfolio URL, and statement of purpose (30 to 300 characters).
- Validate pass tier selection, workshop multi-select group (between 1 and 3 tracks), 5-factor password strength, password matching, and mandatory terms acceptance.
- Display field-level validation errors and overall step error banners with animated visual feedback.
- Allow users to submit valid registrations and view the confirmed attendee directory dynamically.
- Provide interactive confirmation with celebratory confetti and a printable digital attendee badge with QR code.
- Persist confirmed attendee records in browser localStorage using JSON conversion.
- Create a responsive, accessible user-friendly interface for desktop, tablet, and mobile screens.
- Build and deploy the production application using Vite and GitHub Pages.

4. INTRODUCTION

ReactJS is a JavaScript library used to build interactive user interfaces. It organizes an interface into reusable components and updates the rendered page when application state changes. React state and event handlers allow a web application to respond to user input without manually rebuilding the entire page.

EventSphere - Event & Workshop Registration Portal is a single-page front-end application created for Web Technology Practical 3. It provides a multi-step form for entering applicant information, selecting an academic track and institution, selecting ticket tiers and workshop tracks, calculating real-time pricing and promo discounts, and establishing secure credentials. The application validates the entered data before accepting it and presents immediate visual feedback when the form is incomplete or invalid.

The application uses React useState for state management, useMemo for reactive price recalculations, and useEffect for synchronizing theme preferences and application records with localStorage. Vite is used as the development and production build tool. The project is hosted in a public GitHub repository and deployed to GitHub Pages.

5. TECHNOLOGIES USED

Technology	Purpose
ReactJS	Component-based user interface and declarative virtual DOM rendering.
JavaScript (ES6+)	Validation logic, event callbacks, object and array operations, regex, temporal age math.
useState	Manages form data, touched fields, errors, current step, and modal visibility.
useMemo	Computes reactive live pricing, workshop add-ons, and promo code discounts.
useEffect	Synchronizes dark/light theme and persists attendee records to browser localStorage.
HTML5 / JSX	Defines the interface structure using JSX syntax and accessible ARIA attributes.
CSS3	Custom visual styling, layout, CSS variables, dark/light themes, animations, and print stylesheets.
Lucide React	Accessible modern iconography for form fields, alerts, and step indicators.
Canvas-Confetti	Celebratory particle animation triggered upon valid registration submission.
Vite	Development server and optimized production build tool.
npm	Installs project dependencies and runs build/lint/test scripts.
localStorage / JSON	Browser-side persistence of confirmed attendee registration records.
VS Code / Web Browser	Development, editing, testing and demonstration environment.
Git / GitHub	Version control and public source-code hosting.
GitHub Pages	Automated production build and public static-site deployment.

6. CLIENT-SIDE VALIDATION

Field	Validation rule	Example of invalid input
Full name	Trimmed name must contain at least 3 alphabetic characters.	Blank, 'Ab', or containing digits (e.g. 'Aarav12').
Email	Must match valid RFC email pattern user@domain.tld with typo detection.	name@, missing top-level domain, or user@gmai.com.
Mobile number	10-digit Indian mobile pattern beginning with 6-9.	12345, letters, or starts with 0-5.
Date of Birth	Temporal date math; attendee must be at least 18 years old and not in the future.	Future date, or minor under 18 (e.g. 2020-05-10).
Institution	College or organization name must contain at least 3 characters.	Blank or short string (e.g. 'AB').
Portfolio URL	Optional; if supplied, must be valid HTTP or HTTPS URL.	ftp://link or domain without protocol.
Statement / Bio	Must contain between 30 and 300 characters.	Under 30 chars (e.g. 'Too short') or > 300 chars.
Workshops	Must select between 1 and 3 workshop tracks.	0 selected, or > 3 workshops chosen.
Password	Must contain 8+ chars, uppercase, lowercase, number, and special character.	Short password ('pass') or missing symbol.
Confirm password	Must match primary password character-by-character.	Mismatch with primary password.
Terms & conduct	Checkbox must be checked before submission.	Unchecked terms agreement checkbox.

7. TESTING AND VALIDATION (T1 to T14)

Test	Input / Action	Expected Result	Observed Result
T1	Submit all fields empty.	Required-field messages appear; no record is added.	PASS (Verified)
T2	Enter a very short name.	Name validation message appears.	PASS (Verified)
T3	Enter an invalid email address.	Email validation message appears.	PASS (Verified)
T4	Enter an invalid mobile number.	Mobile validation message appears.	PASS (Verified)
T5	Enter date of birth for minor (<18).	Age boundary validation message appears.	PASS (Verified)
T6	Leave college / organization empty.	Institution validation appears.	PASS (Verified)
T7	Enter valid details and submit.	Registration added, confetti and digital pass modal appears.	PASS (Verified)

Test	Input / Action	Expected Result	Observed Result
T8	Leave portfolio URL blank.	Allowed if other fields are valid.	PASS (Verified)
T9	Enter an invalid portfolio URL.	URL protocol validation appears.	PASS (Verified)
T10	Refresh after a successful submission.	Saved registrations remain in browser localStorage.	PASS (Verified)
T11	Filter attendees by pass tier.	Table updates dynamically to matching tier.	PASS (Verified)
T12	Clear and reset the form.	Form fields reset to default.	PASS (Verified)
T13	Delete all records.	Empty-state message appears.	PASS (Verified)
T14	Open at mobile viewport width.	Layout remains readable without horizontal overflow.	PASS (Verified)

8. SCREENSHOTS

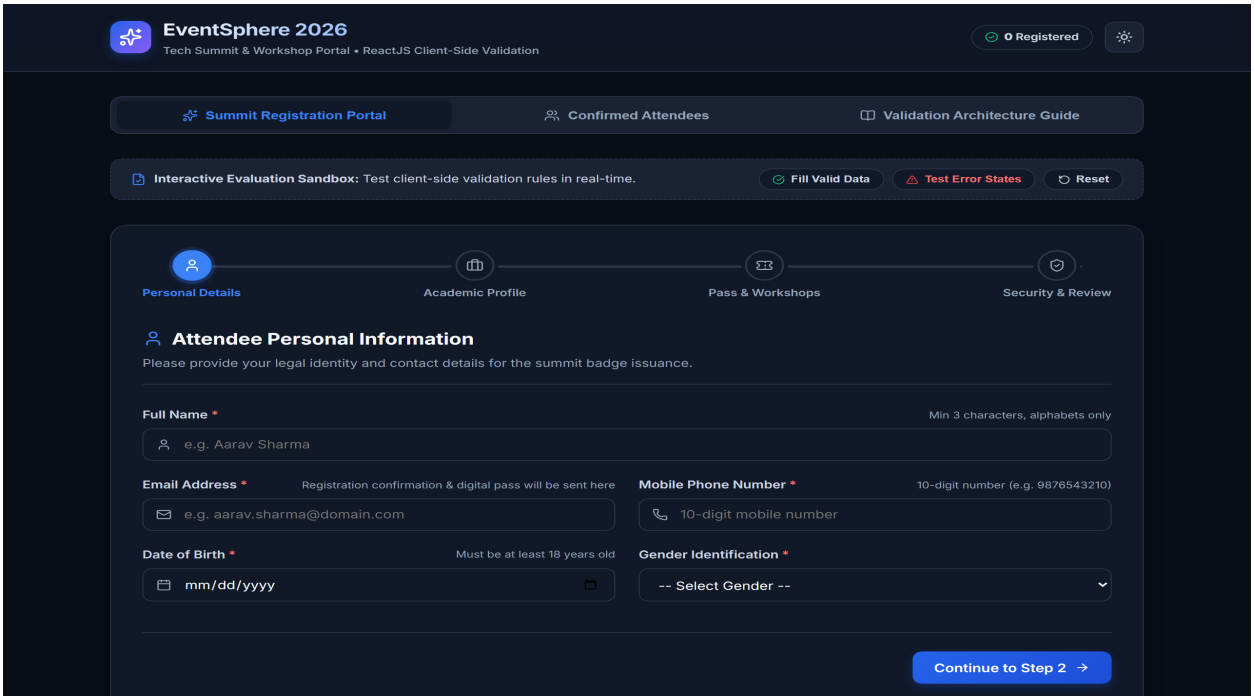


Figure 1: EventSphere desktop landing interface with application form and activity list.

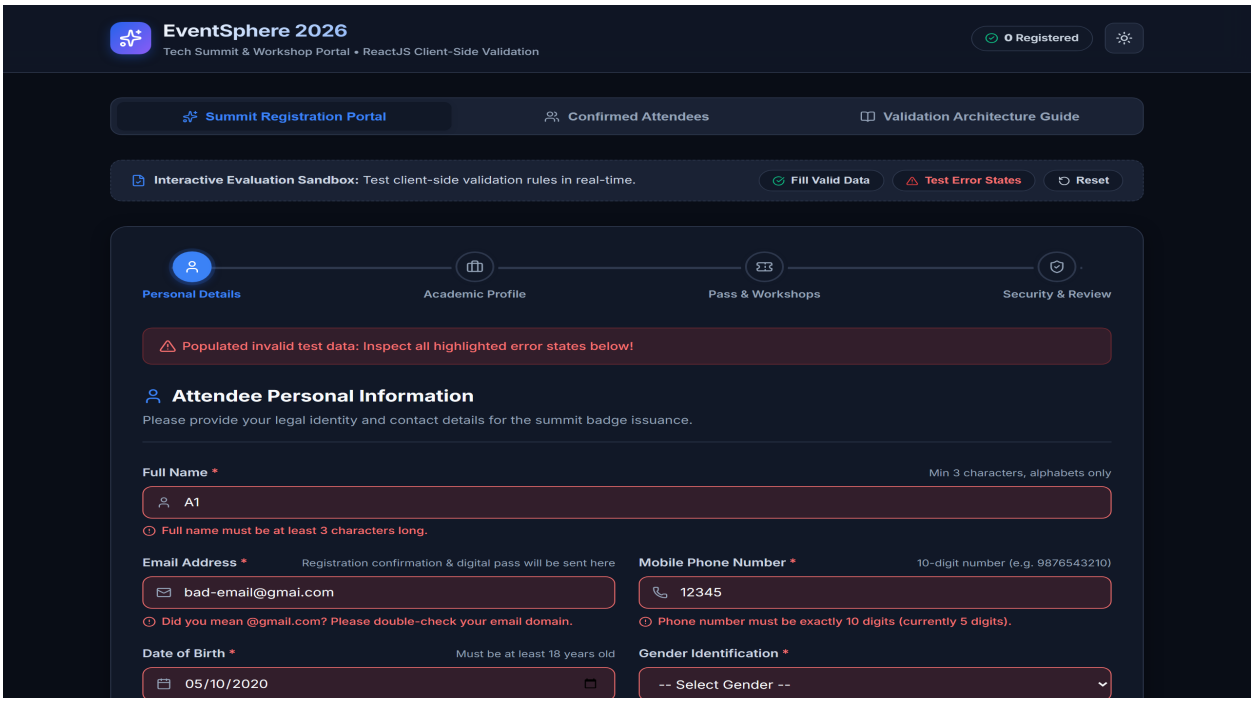


Figure 2: Real-time client-side validation error messages and highlights for incomplete input.

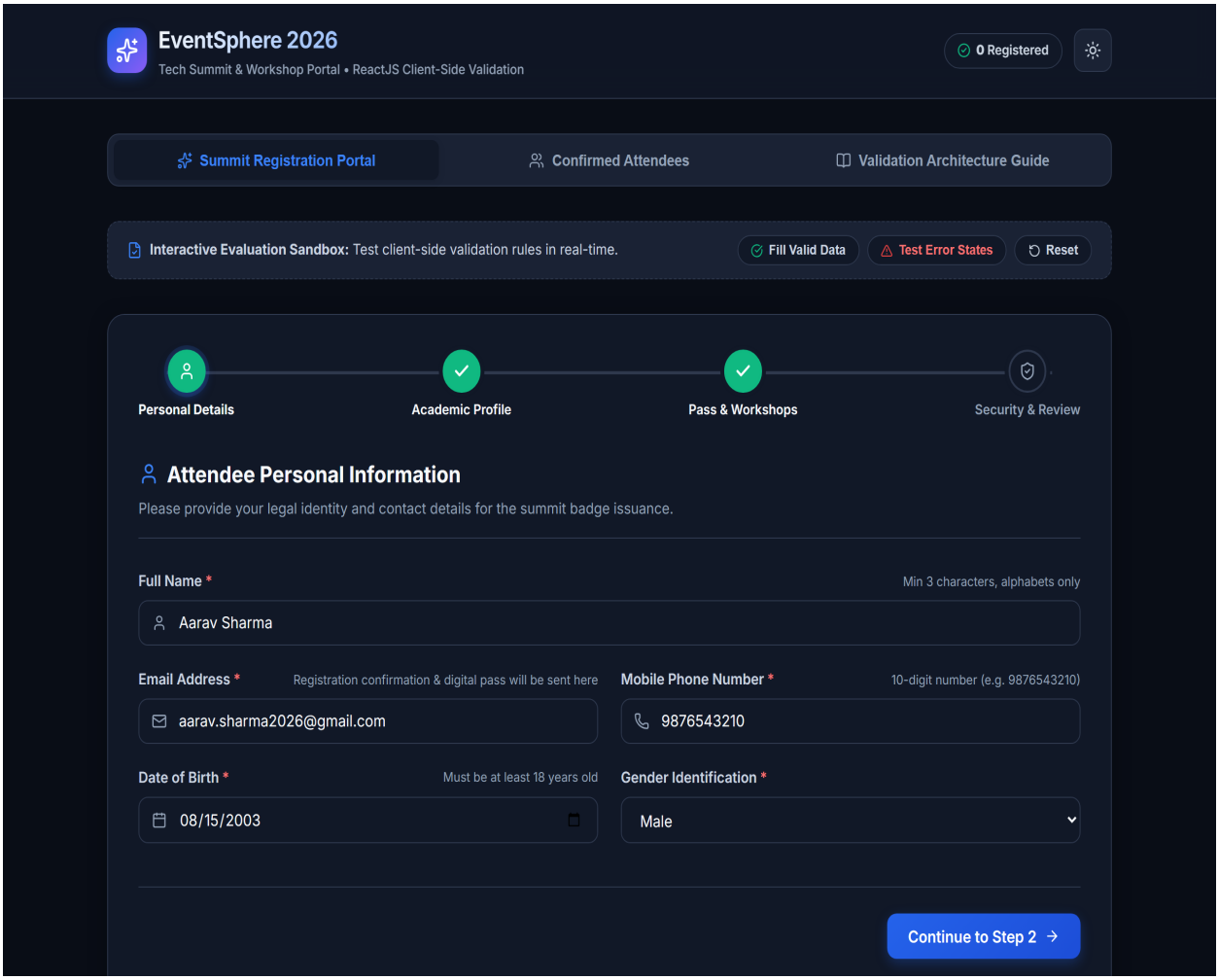


Figure 3: Success state, toast notification, and dynamically rendered digital pass.

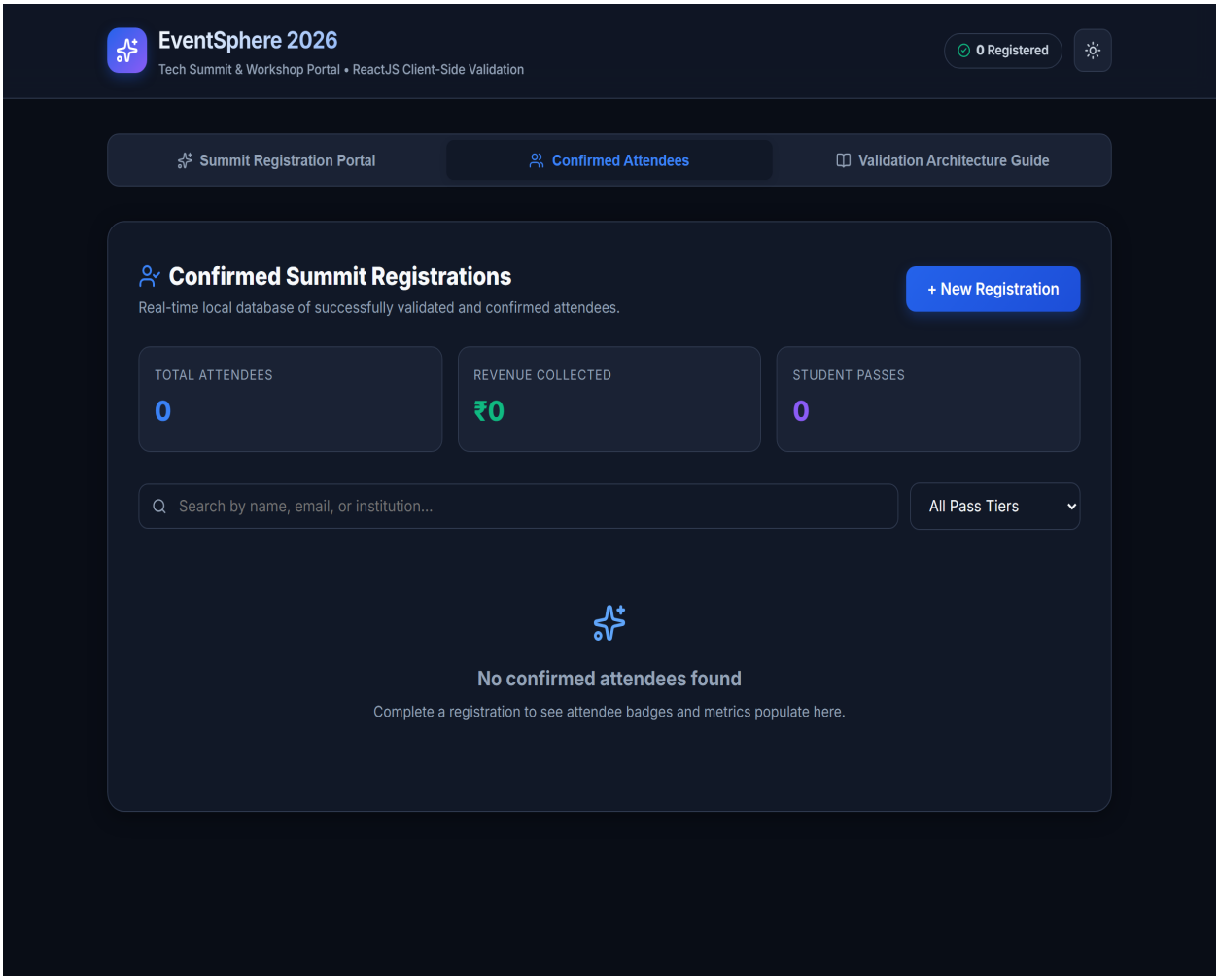


Figure 4: Confirmed Attendees Directory with real-time search and tier filtering.

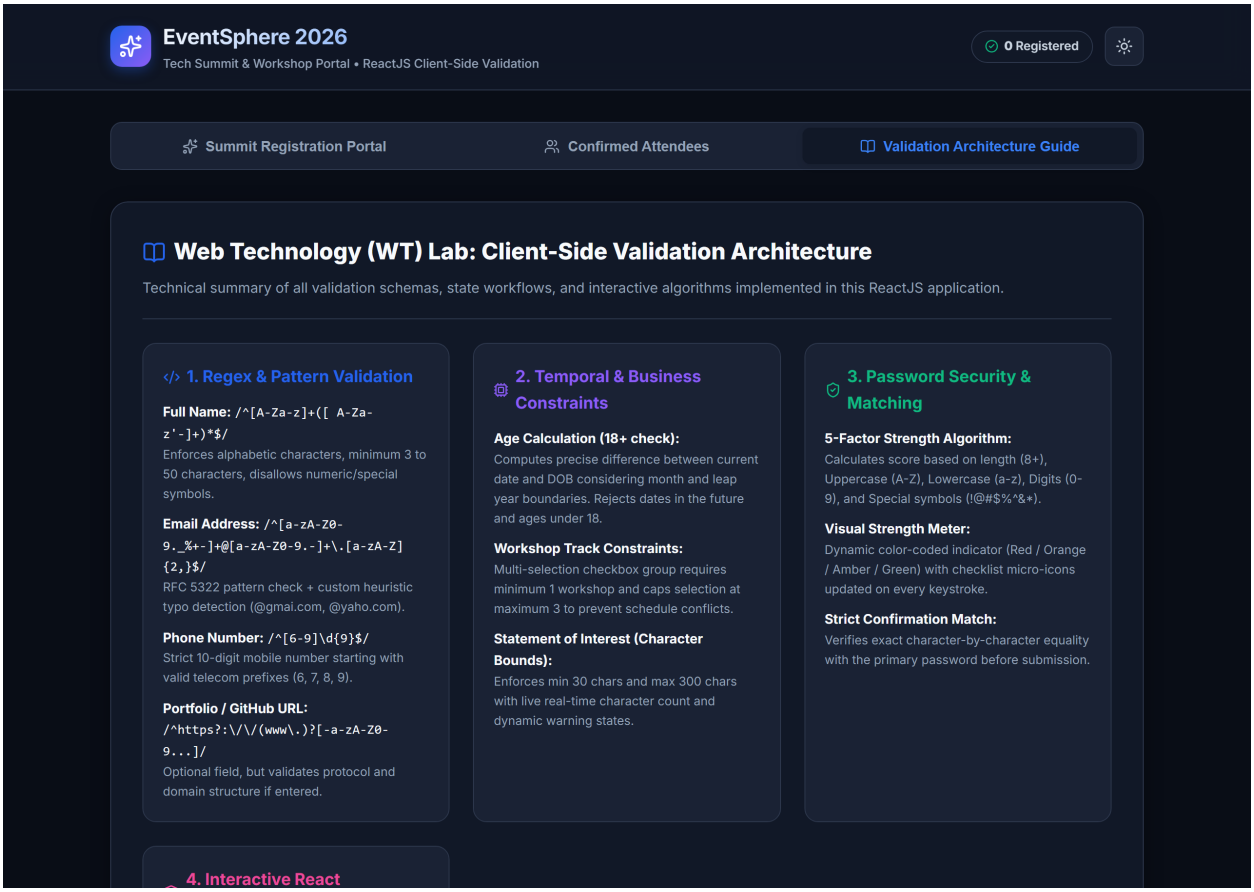


Figure 5: In-app Validation Architecture Guide and rules reference.

9. CONCLUSION

The EventSphere project demonstrates the development of a ReactJS front-end application using JSX, React state, event handling, client-side validation and dynamic rendering. The form validates applicant details before accepting a record, displays useful error feedback for invalid input, and confirms successful submission. Application records are displayed interactively and persisted in localStorage for the same browser.

The project also demonstrates a Vite production build, source-code management through Git and GitHub, and automated deployment through GitHub Actions to GitHub Pages.